

Obsługa ROS Service

Wprowadzenie - polecenia w terminalu

ROS Service jest kolejnym sposobem komunikacji. Zaletą serwisów jest możliwość wysłania zapytania do serwera i otrzymania odpowiedzi zwrotnej. Podobnie jak dla ROS topics należy znać format przekazywanej wiadomości.

Typy wiadomości serwisowych przechowywane są w katalogu srv, a rozszerzenie serwisu to **.srv**. Przykładowa nazwa pliku z wiadomością: **TeleportAbsolute.srv**. **TeleportAbsolute** wskazuje nazwę typu wiadomości, który nie zawiera **.srv**.

Na serwisie o podanej nazwie obsługiwany jest tylko jeden konkretny typ wiadomości serwisowej. Nie można mieszać typów wiadomości.

Struktura wiadomości serwisowej

W formacie wiadomości wyróżnia się podział na:

- wiadomość wysyłaną przez klienta (górną część wiadomości nad znakiem ---)
- odpowiedź serwera (dolną część wiadomości pod znakiem ---)

Pola w wiadomości serwisowej definiowane są tak samo jak przy topicach.

typ_pola nazwa_pola

Z lewej strony należy podać typ pola, a z prawej nazwę pola. Typ pola może być prosty (np. bool, int8, uint16, float32, string, time) lub złożony. Typ pola złożony wykorzystuje wiadomości pochodzące z innych paczek ROS. Można go rozpoznać po następującym zapisie:

nazwa_paczki/nazwa_typu_wiadomosci_ros nazwa_pola

rossrv - komenda do obsługi wiadomości serwisowych (SRV)

Dla obsługi wiadomości serwisowych używane jest polecenie rossrv. Polecenie można wywołać z następującymi parametrami:

```
In [1]: !rossrv --help
```

rossrv is a command-line tool for displaying information about ROS Service types.

Commands:

rossrv show	Show service description
rossrv info	Alias for rossrv show
rossrv list	List all services
rossrv md5	Display service md5sum
rossrv package	List services in a package
rossrv packages	List packages that contain services

Type rossrv <command> -h for more detailed usage

Ogólny sposób wyświetlania informacji ze strukturą wiadomości serwisowej w ROS:

rossrv show *nazwa_paczki/nazwa_typu_wiadomosci_serwisowej*

Wyświetlenie przykładowej struktury wiadomości serwisowej typu **Task** znajdującej się w paczce **pkg_tsr**

```
In [1]: !rossrv show pkg_tsr/Task
```

```
float64 x
float64 y
---
string result
```

Wyświetlenie przykładowej struktury wiadomości serwisowej typu **LoadMap** znajdującej się w paczce **nav_msgs**

```
In [3]: !rossrv show nav_msgs/LoadMap
```

```

string map_url
---
uint8 RESULT_SUCCESS=0
uint8 RESULT_MAP_DOES_NOT_EXIST=1
uint8 RESULT_INVALID_MAP_DATA=2
uint8 RESULT_INVALID_MAP_METADATA=3
uint8 RESULT_UNDEFINED_FAILURE=255
nav_msgs/OccupancyGrid map
  std_msgs/Header header
    uint32 seq
    time stamp
    string frame_id
  nav_msgs/MapMetaData info
    time map_load_time
    float32 resolution
    uint32 width
    uint32 height
    geometry_msgs/Pose origin
    geometry_msgs/Point position

```

Dla wiadomości serwisowej typu **LoadMap**, aby dostać się do pól z odpowiedzi o rozdzielczości mapy (**resolution**) należy dostać się kolejno przez pola **map.info.resolution**.

Sprawdzenie struktury wiadomości serwisowej jest możliwe po przekazaniu nazwy samego typu bez wskazania paczki, z której dany typ pochodzi.

rosmg show *nazwa_typu_wiadomości_serwisowej*

Uwaga

Użycie samej nazwy typu wiadomości serwisowej bez wskazania paczki może spowodować wyświetlenie większej liczby wiadomości serwisowych. W zależności od paczki wiadomości serwisowe mogą się między sobą różnić strukturą i nie są ze sobą zgodne pomimo tej samej nazwy typu.

Dla komunikacji **ROS Services** typ wiadomości serwisowej pomiędzy klientem, a serwerem musi być zgodny zarówno co do nazwy typu jak i paczki, z której pochodzi.

rosservice - komenda do obsługi ROS Services

In [7]: `!rosservice --help`

```

Commands:
  rosservice args print service arguments
  rosservice call call the service with the provided args
  rosservice find find services by service type
  rosservice info print information about service
  rosservice list list active services
  rosservice type print service type
  rosservice uri print service ROSRPC uri

```

Type `rosservice <command> -h` for more detailed usage, e.g. `'rosservice call -h'`

Wyświetlanie listy aktualnie dostępnych serwisów wraz z uruchomioną symulacją turtlesim

In [6]: `!rosservice list`

```

/clear
/kill
/reset
/rosout/get_loggers
/rosout/set_logger_level
/spawn
/turtle1/set_pen
/turtle1/teleport_absolute
/turtle1/teleport_relative
/turtlesim/get_loggers
/turtlesim/set_logger_level

```

Dla aktywnego serwisu o podanej nazwie wyświetla informację o typie wiadomości serwisowej, nodzie udostępniającym dany serwis, przyjmowanych argumentach.

rosservice info *nazwa_serwisu*

In [4]: `!rosservice info /turtle1/teleport_absolute`

```

Node: /turtlesim
URI: rosrpc://localhost:59713
Type: turtlesim/TeleportAbsolute
Args: x y theta

```

Dla aktywnego serwisu o podanej nazwie wyświetla informację tylko o typie wiadomości serwisowej.

rosservice type *nazwa_serwisu*

```
In [35]: !rosservice type spawn
```

```
turtlesim/Spawn
```

Do wysłania zapytania klienta na serwer używane jest polecenie rosservice call. Dla wykorzystywanych serwisów dla symulacji turtlesim wystarczy następująca składnia:

rosservice call *nazwa_serwisu* argument1 argument2 ...

Argumenty mogą być opcjonalne i są zależne od typu wiadomości serwisowej. Przed podaniem kolejnych argumentów warto sprawdzić strukturę wiadomości serwisowej i typ argumentów.

Serwis o nazwie "spawn" jest typu turtlesim/Spawn. Dla zapytania klienta pierwsze 3 argumenty (x, y, theta) są typu float32, a ostatni jest typu string (name). Podanie argumentów spowoduje utworzenie nowego robota żółwia pod warunkiem, że nie był on do tej pory utworzony w symulacji.

```
In [11]: !rosservice type spawn
```

```
turtlesim/Spawn
```

```
In [12]: !rossrv show turtlesim/Spawn
```

```
float32 x
float32 y
float32 theta
string name
---
string name
```

```
In [14]: # Dostępne argumenty w kolejności dla serwisu spawn
!rosservice args /spawn
```

```
x y theta name
```

Uwaga

Współrzędne x,y nowego robota żółwia powinny być z przedziału <0,11>

```
In [13]: !rosservice call spawn 10 10 0 t5
```

```
name: "t5"
```

Zamiast wpisu z jupyter notebook polecenie można wpisać z poziomu terminala. Po wpisaniu "rosservice call spawn" i kliknięciu kilka razy tabulatora wiadomość wraz z argumentami powinna zostać uzupełniona domyślnymi parametrami.

Dostępne serwisy dla turtlesim node

W podstawowym zakresie dostępne są następujące serwisy bez względu na robota:

- /clear - wyczyszczenie narysowanych ścieżek
- /kill - usunięcie robota
- /reset - reset środowiska do stanu początkowego
- /spawn - utworzenie nowego robota

Dla pojedynczego utworzonego robota w przestrzeni nazw na przykładzie **turtle1** dostępne są następujące serwisy:

- /turtle1/set_pen - ustawienie koloru pędzla do rysowania
- /turtle1/teleport_absolute - natychmiastowe przeniesienie robota do wskazanej lokalizacji
- /turtle1/teleport_relative - przeniesienie robota, współrzędne podawane w układzie robota

Klient - Python

Importowanie wiadomości serwisowych na podstawie informacji o ich typie jest następujące:

```
from nazwa_paczki.srv import nazwa_typu_wiadomości_serwisowej
```

Import wiadomości zapytania dla klienta. Na końcu nazwy typu pojawia się dodatkowo **Request**.

```
from nazwa_paczki.srv import nazwa_typu_wiadomości_serwisowejRequest
```

```
In [1]: import rospy
rospy.init_node("unikalna_nazwa_noda_serwisu")
```

Do utworzenia obiektu klienta wykorzystywana jest klasa `ServiceProxy` z biblioteki `rospy`. Przyjmowane kolejno argumenty:

- nazwa serwisu
- nazwa typu wiadomości serwisowej

```
rospy.ServiceProxy(nazwa_serwisu, nazwa_typu_wiadomości_serwisowej)
```

Wysłanie zapytania na serwer.

obiekt_klienta(zapytanie)

Przykład klienta wysyłającego żądanie wyczyszczenia mapy z tras robotów

```
In [38]: #import typu wiadomości serwisowej
from std_srvs.srv import Empty
```

```
In [40]: # Utworzenie klienta "ServiceProxy"
clear_map = rospy.ServiceProxy('clear', Empty)

# Wersja 1
# Wysłanie żądania przez klienta. Dla pustego zapytania argument nie jest
# przekazywany do funkcji
clear_map()

# Wersja 2 - To samo wysłanie zapytania przez klienta tylko z utworzeniem
# pustego zapytania. Następuje dodatkowo dodanie do importu EmptyRequest
from std_srvs.srv import EmptyRequest
request = EmptyRequest()
clear_map(request)
```

Out[40]:

Przykład klienta wysyłającego żądanie z uzupełnianiem żądania klienta. Na przykładzie serwisu tworzącego nowego robota (/spawn).

```
In [ ]: from turtlesim.srv import Spawn
```

```
In [ ]: create_new_robot = rospy.ServiceProxy('spawn', Spawn)
# Wersja 1
# Dla serwisu Spawn konieczne jest podanie kolejnych argumentów. Można je podać jako kolejne
# argumenty w trakcie wysłania zapytania
create_new_robot(3, 3, 0, "t8")

#Wersja 2
# lub można utworzyć wiadomość z zapytaniem
from turtlesim.srv import SpawnRequest
request = SpawnRequest()
request.x = 7
request.y = 10
request.theta = 3.14
request.name = "nowy2"
# zapis odpowiedzi w zmiennej response. response jest typu zwracanego przez serwer.
# Więcej o typach wiadomości zwracanych przez serwer w dalszej części dotyczącej serwera.
response = create_new_robot(request)
print(response)
```

```
In [ ]: # Zamiast oddzielnych importów Spawn i SpawnRequest można zapisać
from turtlesim.srv import Spawn, SpawnRequest
# lub można zaimportować wszystkie wiadomości serwisowe z paczki przy użyciu *
from turtlesim.srv import *
```

Serwer - Python

Import wiadomości odpowiedzi serwera dla klienta. Na końcu nazwy typu pojawia się dodatkowo **Response**.

```
from nazwa_paczki.srv import nazwa_typu_wiadomości_serwisowejResponse
```

```
In [3]: # czyszczenie symulacji
!rosservice call reset
```

Do utworzenia obiektu serwera wykorzystywana jest klasa `Service` z biblioteki `rospy`. Przyjmowane kolejno argumenty:

- nazwa serwisu
- nazwa typu wiadomości serwisowej
- nazwa funkcji callback, która ma zostać wywołana po pojawieniu się żądania od klienta na tym serwisie

`rospy.Service(nazwa_serwisu, nazwa_typu_wiadomości_serwisowej, nazwa_funkcji_callback)`

Funkcja callback wykonuje się wtedy, gdy na serwisie o podanej nazwie pojawia się nowe żądanie. Jako argument do funkcji przekazywana jest wiadomość typu `typRequest`, a funkcja w odpowiedzi zwraca `typResponse`. Poniżej przykład serwera odpowiedzialnego za narysowanie kwadratu robotem `turtle1`.

```
In [2]: from pkg_tsr.srv import Task, TaskResponse
from turtlesim.srv import TeleportAbsolute
from std_srvs.srv import Empty

def draw_square_function(req):
    # req - przekazany argument jest typu pkg_tsr/TaskRequest,
    # req jest zapytaniem wysłanym przez klienta
    width = 5
    print("Init pose x=%s, y=%s" % (req.x, req.y))
    # utworzenie niezbędnych klientów do teleportacji żółwia i narysowania kwadratów
    set_pose = rospy.ServiceProxy('/turtle1/teleport_absolute', TeleportAbsolute)
    clear_map = rospy.ServiceProxy('clear', Empty)
    set_pose(req.x, req.y, 0)
    clear_map()
    set_pose(req.x, req.y + width, 0)
    set_pose(req.x + width, req.y + width, 0)
    set_pose(req.x + width, req.y, 0)
    set_pose(req.x, req.y, 0)

    # response - odpowiedź serwera na zapytanie klienta jest typu pkg_tsr/TaskResponse
    response = TaskResponse()
    response.result = "finished"
    return response
```

```
In [3]: # Utworzenie obiektu serwera. Po otrzymaniu zapytania od klienta na serwerze o nazwie
# "robot_teleport" zostanie wywołana funkcja draw_square_function odpowiedzialna
# za rysowanie kwadratu robotem turtle1.
square_server = rospy.Service('draw_square', Task, draw_square_function)
```

Uwaga

Wyłączenie serwera. Można użyć po wprowadzeniu zmian w funkcji serwisowej, aby nie restartować Kernela Jupyter Notebook. Poniższą metodą `shutdown()` zatrzymać działający serwis.

```
In [8]: square_server.shutdown()
```

Wynik działania powyższego serwisu po wywołaniu klienta

```
In [5]: # Klient do testu powyższego serwera.
draw_square = rospy.ServiceProxy('draw_square', Task)
resp = draw_square(3, 2)
```

Init pose x=3.0, y=2.0

Odczyt otrzymanej wartości z serwera

```
In [7]: # Wyświetlenie opisu rezultatu.
resp.result
```

```
Out[7]: 'finished'
```

Klasa z Klientem i Serwerem - ROS - przykład

Przy tworzeniu obiektu Robota w symulacji pojawia się nowy robot o podanej nazwie w miejscu o podanych współrzędnych. Wysłanie zapytania na server o nazwie "nazwa_robota/draw_square" powoduje narysowanie kwadratu danym robotem. W obiekcie robota wywoływana jest funkcja `_draw_square_function`.

```
In [1]: # Kompletny kod dla robota
```

```

import rospy
from turtlesim.srv import TeleportAbsolute, Spawn
from std_srvs.srv import Empty
from pkg_tsr.srv import Task, TaskResponse
rospy.init_node("unikalna_nazwa_noda_serwisy")

class Robot:
    def __init__(self, name, x, y):
        self.name = name
        # utworzenie serwera /go_to_goal
        self.follow_service = rospy.Service(str(self.name) + '/go_to_goal', Task, self._go_to_goal)
        self.square_service = rospy.Service(str(self.name) + '/draw_square', Task, self._draw_square_fun)
        self.create_robot(x,y)

    def create_robot(self, x, y):
        # Użycie istniejącego klienta do utworzenia robota w symulacji
        create_new_turtle = rospy.ServiceProxy('spawn', Spawn)
        create_new_turtle(x, y, 0, self.name)

    def start(self):
        # Wznowienie działania serwisów dla robota.
        self.follow_service = rospy.Service(str(self.name) + '/go_to_goal', Task, self._go_to_goal)
        self.square_service = rospy.Service(str(self.name) + '/draw_square', Task, self._draw_square_fun)

    def stop(self):
        # Zatrzymanie aktywnych serwisów dla robota.
        self.follow_service.shutdown()
        self.square_service.shutdown()

    def _go_to_goal(self, msg_srv):
        """
        Tutaj powinien znaleźć się program obsługujący działanie serwera /go_to_goal.
        Po wysłaniu zapytania przez dowolnego klienta na ten serwer powoduje on, że
        robot realizuje zadanie dojazdu do punktu. W zależności od tego czy udało się
        zrealizować dojazd czy nie zwracany jest odpowiedni wynik. Dla kilku robotów
        nazwy serwerów powinny być związane z nazwami różnych robotów.
        """
        # Utworzenie i uzupełnienie odpowiedzi odpowiednimi wartościami.
        response = TaskResponse()

        # zwrócenie wyniku
        return response

    def _draw_square_function(self, req):
        # req - przekazany argument jest typu pkg_tsr/TaskRequest,
        # req jest zapytaniem wysłanym przez klienta
        width = 5
        print("Init pose x=%s, y=%s" % (req.x, req.y))
        # utworzenie niezbędnych klientów do teleportacji żółwia i narysowania kwadratów
        set_pose = rospy.ServiceProxy('/turtle1/teleport_absolute', TeleportAbsolute)
        clear_map = rospy.ServiceProxy('clear', Empty)
        set_pose(req.x, req.y, 0)
        clear_map()
        set_pose(req.x, req.y + width, 0)
        set_pose(req.x + width, req.y + width, 0)
        set_pose(req.x + width, req.y, 0)
        set_pose(req.x, req.y, 0)

        # response - odpowiedź serwera na zapytanie klienta jest typu pkg_tsr/TaskResponse
        response = TaskResponse()
        response.result = "finished"
        return response

turtle2 = Robot("turtle2", 1, 1)

```

```
In [3]: turtle2.stop()
```

```
In [5]: turtle2.start()
```

```
In [6]: # Klient do testu serwera rysowania kwadratu dla robota.
draw_square = rospy.ServiceProxy('turtle2/draw_square', Task)
resp = draw_square(1, 2)
```

```
Init pose x=1.0, y=2.0
```

```
In [ ]:
```